

Smart Asset Management and Resource Allocation Platform

Participation Format

This is a team problem statement.

Teams may consist of up to the maximum number of 03 participants.

Participants are expected to design and develop a full-stack asset management platform that enables efficient inventory tracking, resource allocation, and operational visibility.

The challenge evaluates software engineering, database design, system architecture, API development, and user experience design skills.

1. Motivation

Organizations of all sizes rely on efficient asset management systems to track inventory, allocate resources, monitor utilization, and maintain accountability.

From universities and event management companies to logistics providers and enterprises, poor inventory visibility often results in scheduling conflicts, resource underutilization, delayed returns, and operational inefficiencies.

The Cultural Council of IIT Roorkee faces a similar challenge while managing a large pool of shared resources such as:

- DSLR Cameras
- Studio Lighting Equipment
- Audio Systems
- Costumes
- Stage Props
- Recording Equipment
- Event Infrastructure

These assets are frequently shared across multiple sections and events, making efficient resource allocation critical.

Currently, resource management often relies on fragmented communication channels, spreadsheets, manual registers, and informal coordination processes.

This challenge provides an opportunity to build a real-world asset management platform that mirrors problems commonly encountered in inventory management, resource planning, logistics operations, and enterprise software systems.

2. Problem Statement

Design and develop a full-stack Asset Management and Resource Allocation Platform that enables organizations to efficiently manage shared resources.

The platform should provide a centralized system for:

- Inventory management
- Asset booking
- Approval workflows
- Resource allocation
- Utilization tracking
- Operational analytics

The system should support role-based workflows for administrators and resource consumers while maintaining simplicity, usability, and scalability.

Participants should focus on building a robust and user-friendly solution capable of handling real-world asset management scenarios.

3. Functional Requirements

Mandatory Features

A. User Authentication

Implement a secure authentication system.

Users should be able to:

- Register and log in
- Access role-specific functionality
- Maintain secure sessions

Participants may use:

- JWT Authentication
 - Session-Based Authentication
 - OAuth (Optional)
-

B. Inventory Management

Administrators should be able to:

- Add assets
- Edit asset information
- Delete assets
- Categorize assets
- Manage available quantities

Each asset should contain information such as:

- Asset Name
 - Category
 - Description
 - Quantity Available
 - Status
-

C. Asset Discovery and Booking

Users should be able to:

- Browse available assets
- Search and filter inventory
- View asset availability
- Request assets for specific durations

The platform should prevent booking requests exceeding available inventory.

D. Approval Workflow

Administrators should be able to:

- Review booking requests
- Approve requests
- Reject requests
- View active allocations

Users should be able to track the status of their requests.

E. Asset Issue and Return Management

The platform should support:

- Asset issuance
- Asset return tracking
- Due date management
- Asset status updates

The system should maintain accurate inventory counts throughout the asset lifecycle.

F. Analytics Dashboard

Build a dashboard providing insights such as:

- Most frequently utilized assets
- Asset utilization rates
- Active bookings
- Available inventory
- Overdue returns

Visualizations may include:

- Bar Charts
 - Pie Charts
 - Line Graphs
 - Summary Cards
-

G. Borrowing History

Users should be able to:

- View their borrowing history
- Track active bookings
- Monitor past requests

Administrators should be able to view system-wide activity.

Optional Features

A. Notification System

Notify users regarding:

- Booking approvals
- Booking rejections
- Upcoming return deadlines
- Overdue assets

Notifications may be delivered through:

- Email
 - In-App Notifications
-

B. Audit Logs

Track important actions such as:

- Asset creation
 - Inventory updates
 - Booking approvals
 - Asset returns
-

C. QR Code-Based Asset Operations

Allow administrators to:

- Generate QR codes for assets
- Scan assets during issue and return

- View asset information instantly
-

D. Asset Health Tracking

Track:

- Asset condition
 - Maintenance history
 - Damage reports
-

E. Dockerized Deployment

Provide a containerized development environment using Docker and Docker Compose.

4. Deliverables

Deliverable 1: Working Application

A functional application demonstrating the implemented features.

Deliverable 2: Design Document

Format: PDF

The document should include:

- Problem Understanding
- System Architecture
- Database Schema
- Entity Relationship Diagram (ERD)
- API Overview
- Design Decisions

Maximum Length: 8 Pages

Deliverable 3: Public GitHub Repository

The repository should contain:

- Source Code
- Configuration Files
- Database Scripts
- Documentation

The project should be reproducible.

Deliverable 4: README.md

The README should include:

- Project Overview
 - Technology Stack
 - Setup Instructions
 - Running the Application
 - Feature List
-

Deliverable 5: Demonstration Video

Duration: Maximum 3 Minutes

The video should demonstrate:

- User Authentication
 - Asset Booking Workflow
 - Approval Process
 - Dashboard Functionality
 - Key Technical Features
-

5. Guidelines

1. Prioritize Core Functionality

A well-executed booking and inventory management system is preferable to an incomplete feature-heavy platform.

2. Focus on User Experience

The platform should be intuitive and easy to use for both administrators and users.

3. Maintain Data Integrity

Inventory counts, bookings, and returns should remain consistent throughout the system.

4. Design for Scalability

Consider how the system would support larger inventories and increased user activity.

5. Follow Good Engineering Practices

Strong submissions should demonstrate:

- Clean Code
 - Modular Design
 - Proper Documentation
 - Maintainability
-

6. Evaluation Criteria

Criterion	Weightage
Functionality & Feature Completeness	30%
System Design & Architecture	20%

Database Design & Backend Logic	15%
User Experience & Interface Design	15%
Code Quality & Documentation	10%
Innovation & Additional Features	10%

Bonus Considerations

Additional credit may be awarded for:

- Notification Systems
 - QR-Based Operations
 - Audit Logging
 - Asset Health Tracking
 - Dockerized Deployment
 - Advanced Analytics
-

Type your text